DATA 255 Deep Learning and Computer Vision in Applied Data Intelligence

March 4, 2026

Taehee Jeong, Ph.D.

SJSU SAN JOSÉ STATE UNIVERSITY

Today’s Topics(1)

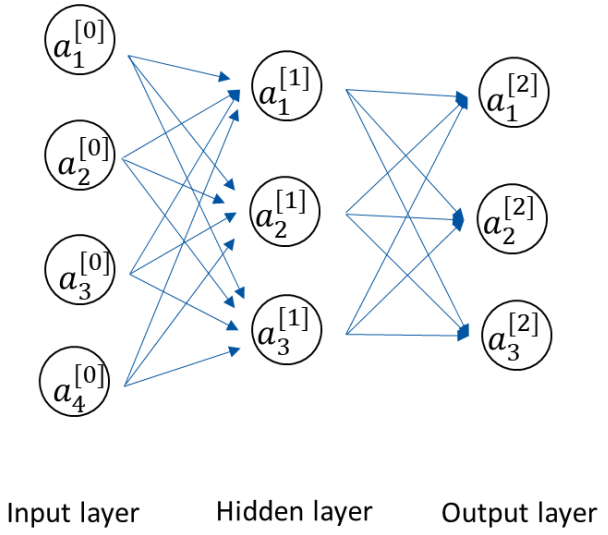
- Deep neural network (DNN) vs. Convolutional neural network (CNN)
- Convolution layer
- Convolution (CONV)

Deep neural network (DNN)

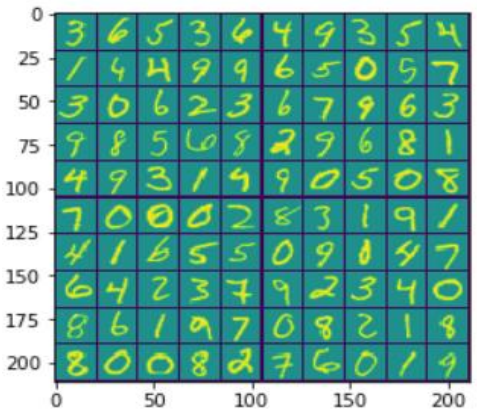
; Artificial neural network (ANN)

It can be applied to regression or classification.

- Regression: predict continuous valued output
- Classification: predict categorical value



Digital representation of an image

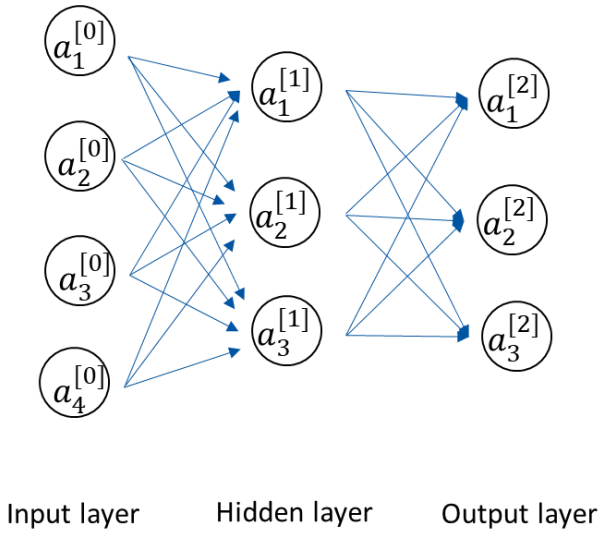


	0	1	2	3	4	5	6	7	8	9	...	391	392	393	394	395	396	397	398	399	y
0	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	5
1	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	9
2	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	7
3	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	6
4	0	0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0	5

- Grayscale image is a matrix of pixels (**pic**ture **el**ements)
- Dimensions of this matrix are called image resolution (e.g. 28 x 28)
- Each pixel stores its brightness (or **intensity**) ranging from 0 to 255, 0 intensity corresponds to black color
- Color images store pixel intensities for 3 channels: red, green and blue

Why not Deep Neural Network?

1000 x 1000 x 3 = 3M



Why not Deep Neural Network?

- Why not simply use a regular deep neural network with fully connected layers for image recognition tasks?
- Unfortunately, although this works fine for small images (e.g., MNIST), it breaks down for larger images because of the huge number of parameters it requires.
- For example, a 100 × 100 image has 10,000 pixels, and if the first layer has just 1,000 neurons (which already severely restricts the amount of information transmitted to the next layer),
- this means a total of 10 million connections. And that's just the first layer.
- CNNs solve this problem using partially connected layers.

Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

Convolutional Layer

- The most important building block of a CNN is the *convolutional layer*:
- Neurons in the first convolutional layer are not connected to every single pixel in the input image, but only to pixels in their receptive fields.
- Neuron in the second convolutional layer is connected only to neurons located within a small rectangle in the first layer.
- This architecture allows the network to concentrate on low-level features in the first hidden layer, then assemble them into higher-level features in the next hidden layer, and so on.
- This hierarchical structure is common in real-world images, which is one of the reasons why CNNs work so well for image recognition.

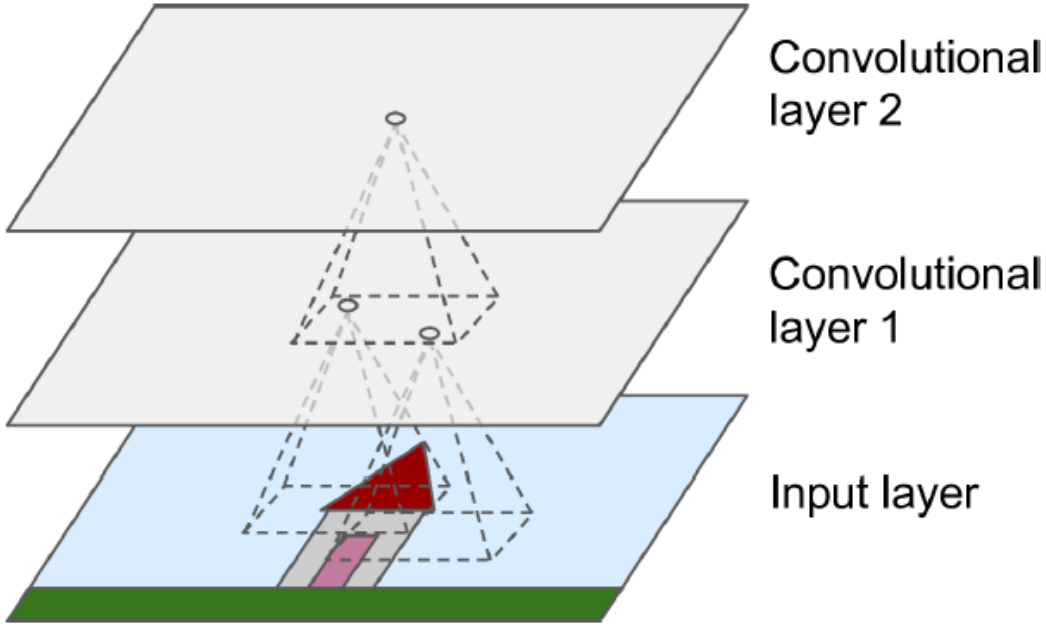
Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

Advantages of CNN over DNN for image classification

- CNN has **fewer parameters** than a fully connected DNN
- CNN can detect a particular **feature anywhere** on the image instead of detecting only in that **particular location**.
- CNN's architecture is able to identify pixels' organization (how each pixel is located).

Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

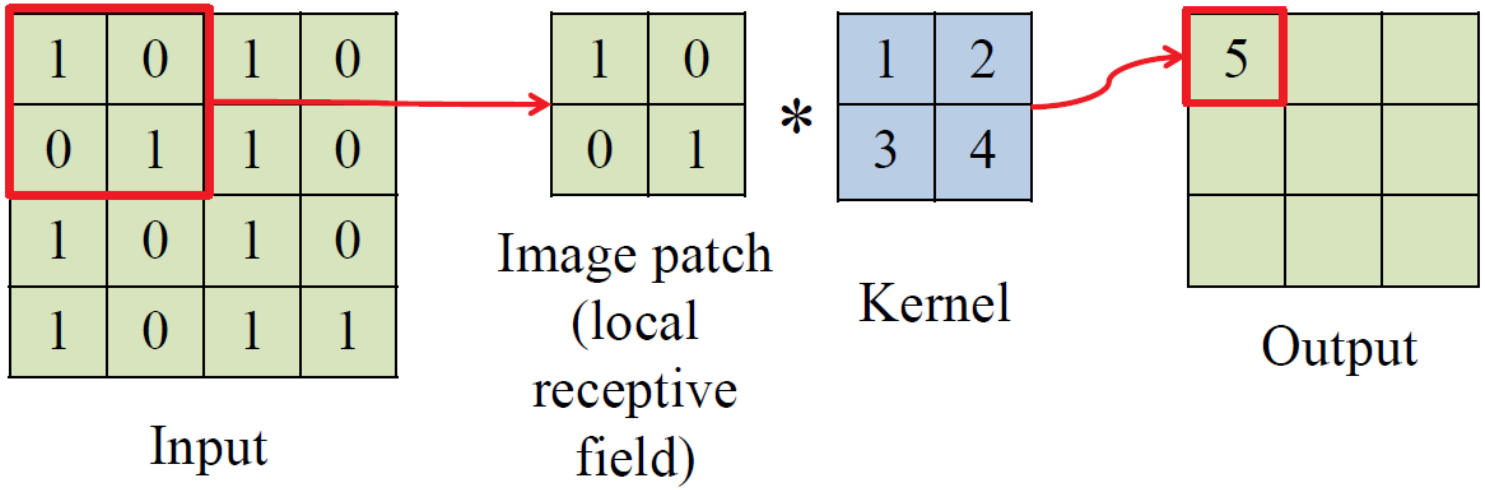
Convolutional layer



Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

Convolutions

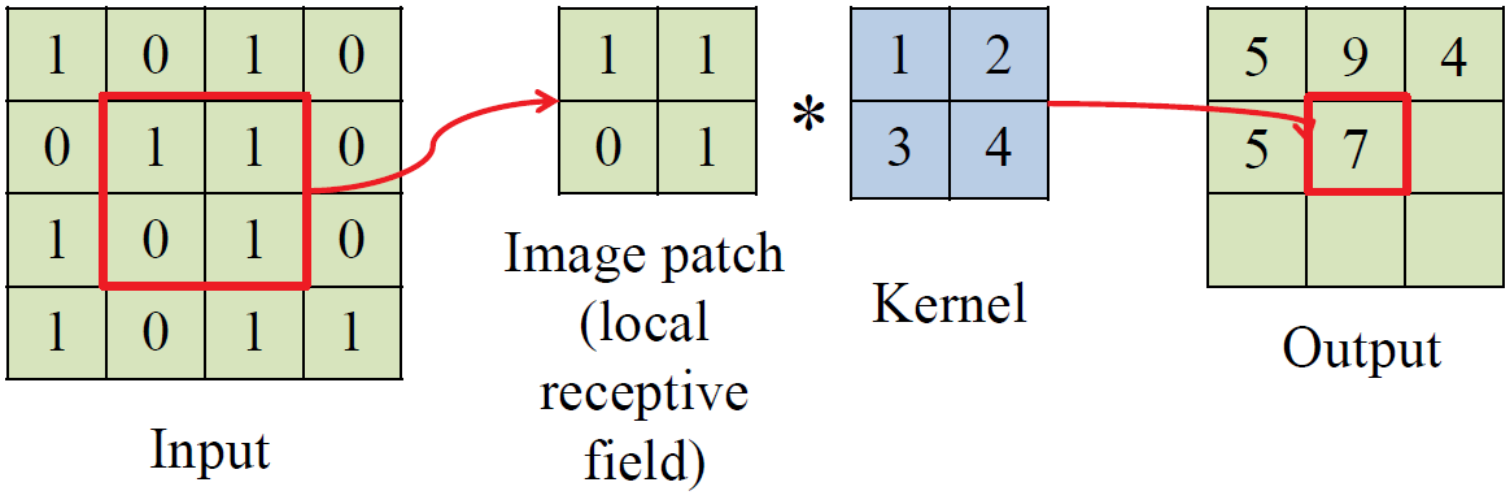
Convolution is a dot product of a **kernel** (or filter)
and a patch of an image (**local receptive field**) of the same size



Source: Intro to deep learning, national research university

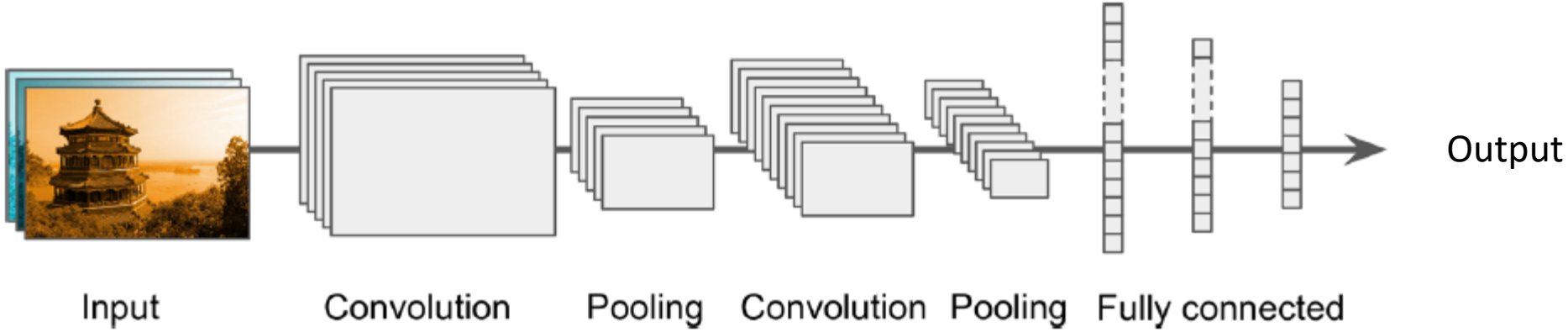
Convolutions

Convolution is a dot product of a **kernel** (or filter)
and a patch of an image (**local receptive field**) of the same size



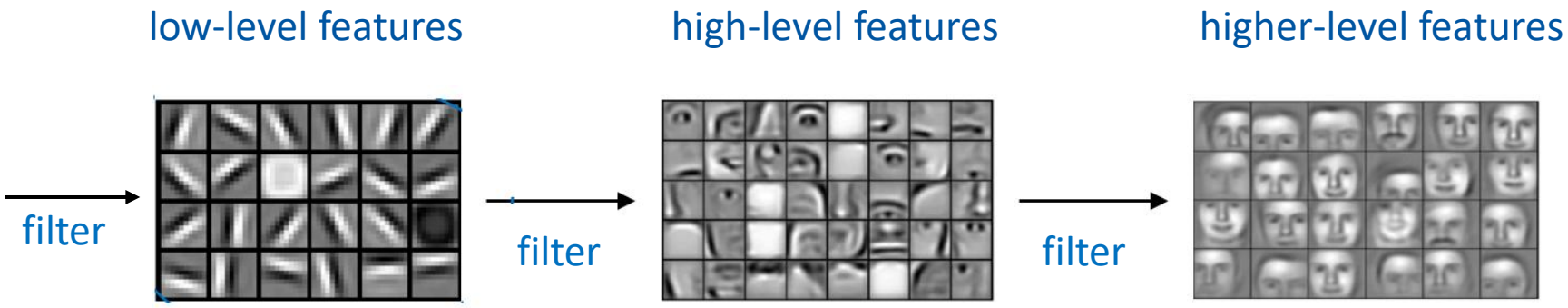
Source: Intro to deep learning, national research university

CNN Architecture



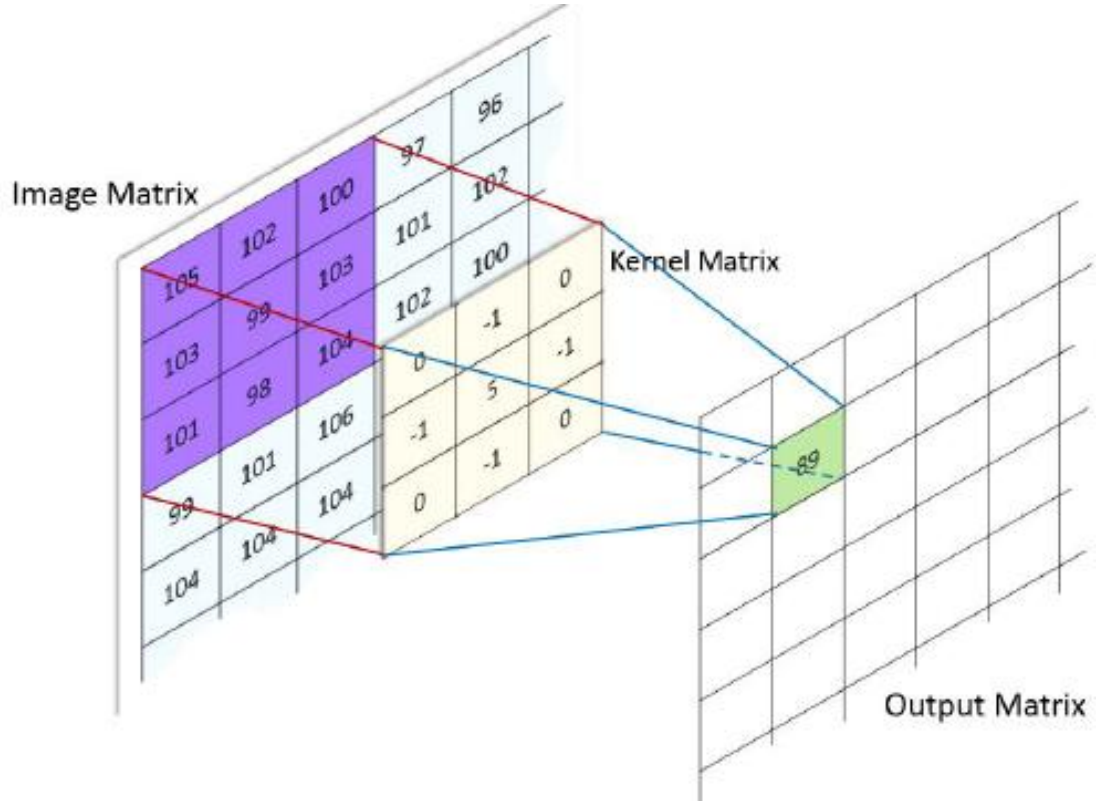
Source: *Hands-On Machine Learning with Scikit-Learn & TensorFlow*

Convolutional layer



Source: Deep Neural Network, Andrew Ng

Image convolution operation



Source: Machinelearninguru.com/computer_vision/basics/convolution/image_convolution_1.html

Image convolution operation

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

105	102	100	97	96
103	99	103	101	102
101	98	104	102	100
99	101	106	104	99
104	104	104	100	98

Image Matrix

$$105 * 0 + 102 * -1 + 100 * 0$$
$$+103 * -1 + 99 * 5 + 103 * -1$$
$$+101 * 0 + 98 * -1 + 104 * 0 \quad = 89$$

	89			

Output Matrix

Source: Machinelearninguru.com/computer_vision/basics/convolution/image_convolution_1.html

Image convolution operation

Kernel Matrix

105	102	100	97	96
103	99	103	101	102
101	98	104	102	100
99	101	106	104	99
104	104	104	100	98

Image Matrix

0	-1	0
-1	5	-1
0	-1	0

	89	111		

Output Matrix

$$102 * 0 + 100 * -1 + 97 * 0$$
$$+99 * -1 + 103 * 5 + 101 * -1$$
$$+98 * 0 + 104 * -1 + 102 * 0 = 111$$

Source: Machinelearninguru.com/computer_vision/basics/convolution/image_convolution_1.html

Convolution operation before activation

X_1	X_2	X_3	X_4
X_5	X_6	X_7	X_8
X_9	X_{10}	X_{11}	X_{12}
X_{13}	X_{14}	X_{15}	X_{16}

*

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

=

Z_1	Z_2
Z_3	Z_4

$$Z_1 = X_1 \times W_1 + X_2 \times W_2 + X_3 \times W_3 + X_5 \times W_4 + X_6 \times W_5 + X_7 \times W_6 + X_9 \times W_7 + X_{10} \times W_8 + X_{11} \times W_9$$

$$Z_2 = X_2 \times W_1 + X_3 \times W_2 + X_4 \times W_3 + X_6 \times W_4 + X_7 \times W_5 + X_8 \times W_6 + X_{10} \times W_7 + X_{11} \times W_8 + X_{12} \times W_9$$

$$Z_3 = X_5 \times W_1 + X_6 \times W_2 + X_7 \times W_3 + X_9 \times W_4 + X_{10} \times W_5 + X_{11} \times W_6 + X_{13} \times W_7 + X_{10} \times W_8 + X_{15} \times W_9$$

$$Z_4 = X_6 \times W_1 + X_7 \times W_2 + X_8 \times W_3 + X_{10} \times W_4 + X_{11} \times W_5 + X_{12} \times W_6 + X_{14} \times W_7 + X_{15} \times W_8 + X_{16} \times W_9$$

Convolution operation in activation

Input matrix

X_1	X_2	X_3	X_4
X_5	X_6	X_7	X_8
X_9	X_{10}	X_{11}	X_{12}
X_{13}	X_{14}	X_{15}	X_{16}

Size: 4 x 4

convolution

*

filter

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

Size: 3 x 3

g : activation function

b : bias

=

$g(Z_1 + b)$	$g(Z_2 + b)$
$g(Z_3 + b)$	$g(Z_4 + b)$

=

Output matrix

a_1	a_2
a_3	a_4

Size: 2 x 2

Today's Topics(2)

- Operations in CNN
  - Convolution (CONV)
  - Padding
  - Stride
  - Pooling (POOL)
  - Fully connected (FC)

Edge pixels

Input matrix

X_1	X_2	X_3	X_4
X_5	X_6	X_7	X_8
X_9	X_{10}	X_{11}	X_{12}
X_{13}	X_{14}	X_{15}	X_{16}

convolution

*

filter

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

What about information from edge pixels?

Padding

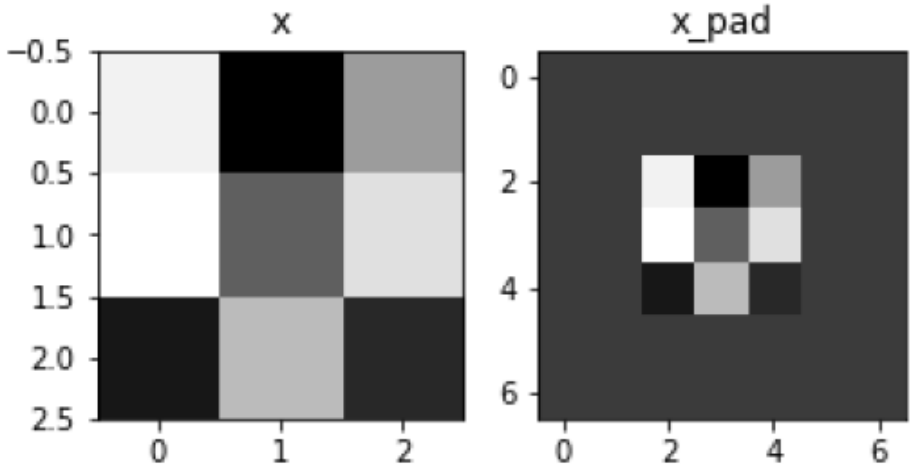
To detect the information of edge or border

To prevent shrinking(compression) of size of next layer

Zero pad

Input: (m, n_H, n_W, n_C)

Output: (m, n_H + 2*pad, n_W + 2*pad, n_C)



Source: Deep Learning, Andrew Ng

Zero pad

np.pad

```
In [4]: a

Out[4]: array([[0, 1, 2],
               [3, 4, 5]])
```

```
In [7]: np.pad(a, (1,1), 'constant', constant_values=(0))

Out[7]: array([[0, 0, 0, 0, 0],
               [0, 0, 1, 2, 0],
               [0, 3, 4, 5, 0],
               [0, 0, 0, 0, 0]])
```

```
In [8]: np.pad(a, (2,2), 'constant', constant_values=(0))

Out[8]: array([[0, 0, 0, 0, 0, 0, 0],
               [0, 0, 0, 0, 0, 0, 0],
               [0, 0, 0, 1, 2, 0, 0],
               [0, 0, 3, 4, 5, 0, 0],
               [0, 0, 0, 0, 0, 0, 0],
               [0, 0, 0, 0, 0, 0, 0]])
```

Padding

0	0	0	0	0	0
0	X_1	X_2	X_3	X_4	0
0	X_5	X_6	X_7	X_8	0
0	X_9	X_{10}	X_{11}	X_{12}	0
0	X_{13}	X_{14}	X_{15}	X_{16}	0
0	0	0	0	0	0

*

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

=

Z_1	Z_2	Z_3	Z_4
Z_5	Z_6	Z_7	Z_8
Z_9	Z_{10}	Z_{11}	Z_{12}
Z_{13}	Z_{14}	Z_{15}	Z_{16}

Convolution operation on borders

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

210	89	111		

Output Matrix

$$0 * 0 + 105 * -1 + 102 * 0$$
$$+0 * -1 + 103 * 5 + 99 * -1$$
$$+0 * 0 + 101 * -1 + 98 * 0 \quad = 210$$

Source: Machinelearninguru.com/computer_vision/basics/convolution/image_convolution_1.html

Convolution operation on borders

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

320				
210	89	111		

Output Matrix

$$0*0+0*-1+0*0$$
$$+0*-1+105*5+102*-1$$
$$+0*0+103*-1+99*0 \quad = 320$$

Source: Machinelearninguru.com/computer_vision/basics/convolution/image_convolution_1.html

Padding: 1 zero padding

0	0	0	0	0	0
0	X_1	X_2	X_3	X_4	0
0	X_5	X_6	X_7	X_8	0
0	X_9	X_{10}	X_{11}	X_{12}	0
0	X_{13}	X_{14}	X_{15}	X_{16}	0
0	0	0	0	0	0

*

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

=

Z_1	Z_2	Z_3	Z_4
Z_5	Z_6	Z_7	Z_8
Z_9	Z_{10}	Z_{11}	Z_{12}
Z_{13}	Z_{14}	Z_{15}	Z_{16}

$$Z_1 = 0 \times W_1 + 0 \times W_2 + 0 \times W_3 + 0 \times W_4 + X_1 \times W_5 + X_2 \times W_6 + 0 \times W_7 + X_5 \times W_8 + X_6 \times W_9$$

$$Z_4 = 0 \times W_1 + 0 \times W_2 + 0 \times W_3 + X_3 \times W_4 + X_4 \times W_5 + 0 \times W_6 + X_7 \times W_7 + X_8 \times W_8 + 0 \times W_9$$

Padding: 2 zero padding

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	X_1	X_2	X_3	X_4	0	0
0	0	X_5	X_6	X_7	X_8	0	0
0	0	X_9	X_{10}	X_{11}	X_{12}	0	0
0	0	X_{13}	X_{14}	X_{15}	X_{16}	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

*

W_1	W_2	W_3
W_4	W_5	W_6
W_7	W_8	W_9

Valid vs. Same convolution

- Valid convolution: without padding

(n x n) * (f x f) → (n-f+1) x (n-f+1)

(6 x 6) * (3x3) → (4 x 4)

- Same convolution: with zero padding, output size is the same as the input size

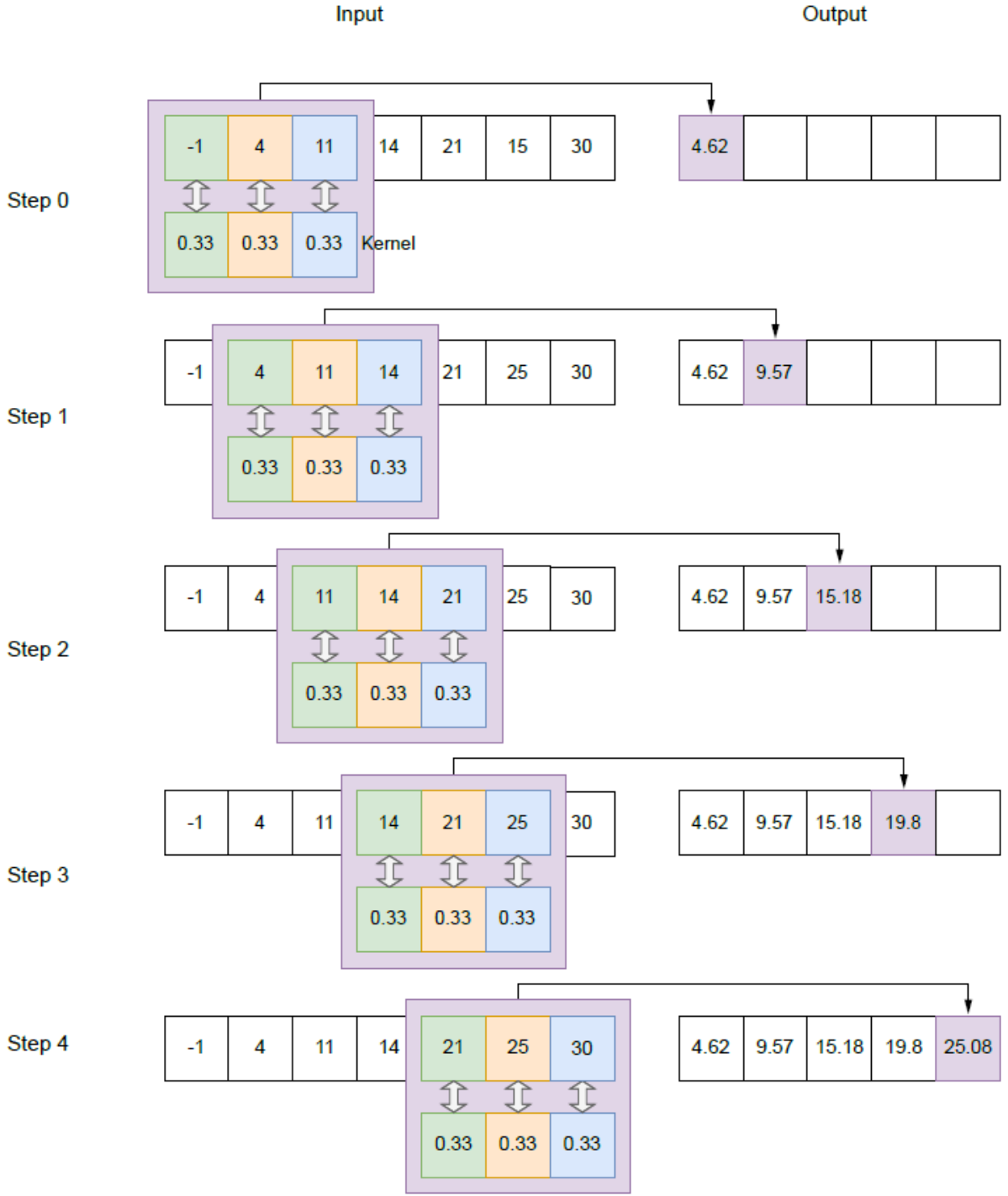
(n x n) * (f x f) → (n+2p-f+1) x (n+2p-f+1)

(6 x 6) * (3x3) → (6 x 6)

Stride

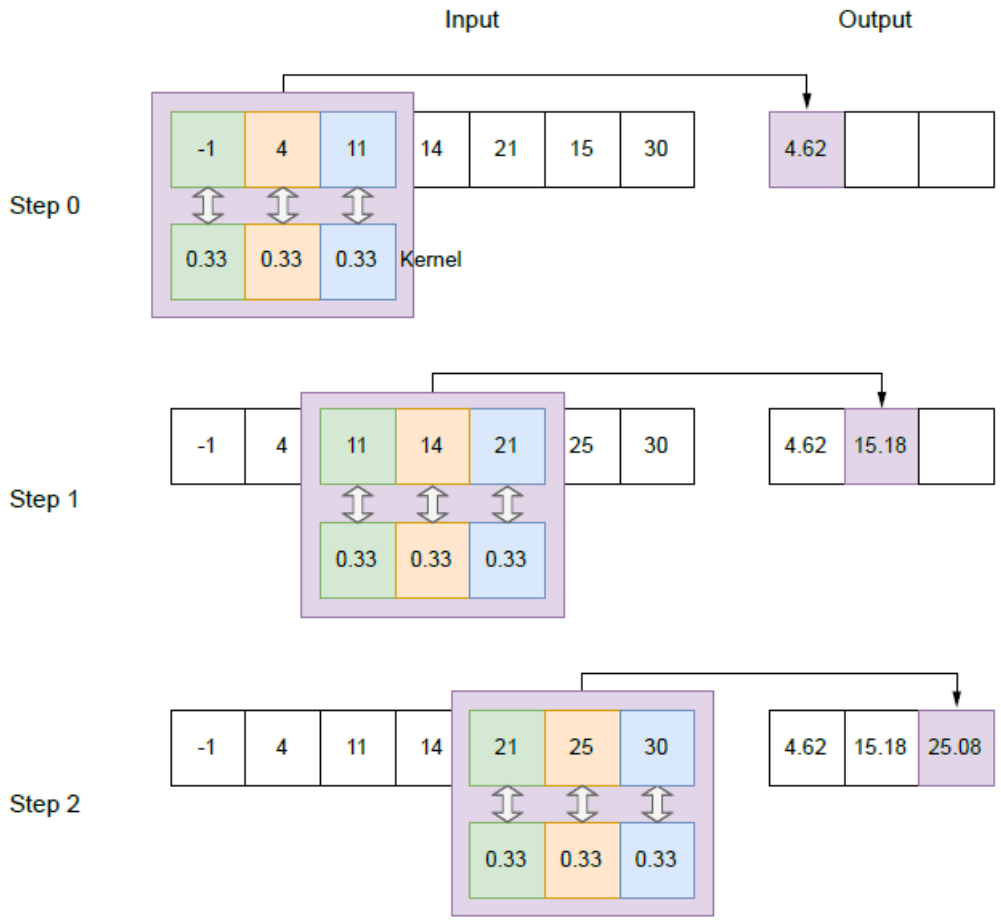
To compress the size of next layer

1D convolution with stride 1



Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

1D convolution with stride 2



Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

SJSU SAN JOSÉ STATE UNIVERSITY

Convolution operation with stride 1

0	0	0	0	0	0
0	105	102	100	97	96
0	103	99	103	101	102
0	101	98	104	102	100
0	99	101	106	104	99
0	104	104	104	100	98

Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

320	206	198	188	
210	89	111		

Output Matrix

$$102 * 0 + 100 * -1 + 97 * 0$$
$$+99 * -1 + 103 * 5 + 101 * -1$$
$$+98 * 0 + 104 * -1 + 102 * 0 = 111$$

Source: Machinelearninguru.com/computer_vision/basics/convolution/convolution_layer.html

Convolution operation with stride 2

0	0	0	0	0	0	0
0	105	102	100	97	96	
0	103	99	103	101	102	1
0	101	98	104	102	100	1
0	99	101	106	104	99	
0	104	104	104	100	98	1

Image Matrix

Kernel Matrix

0	-1	0
-1	5	-1
0	-1	0

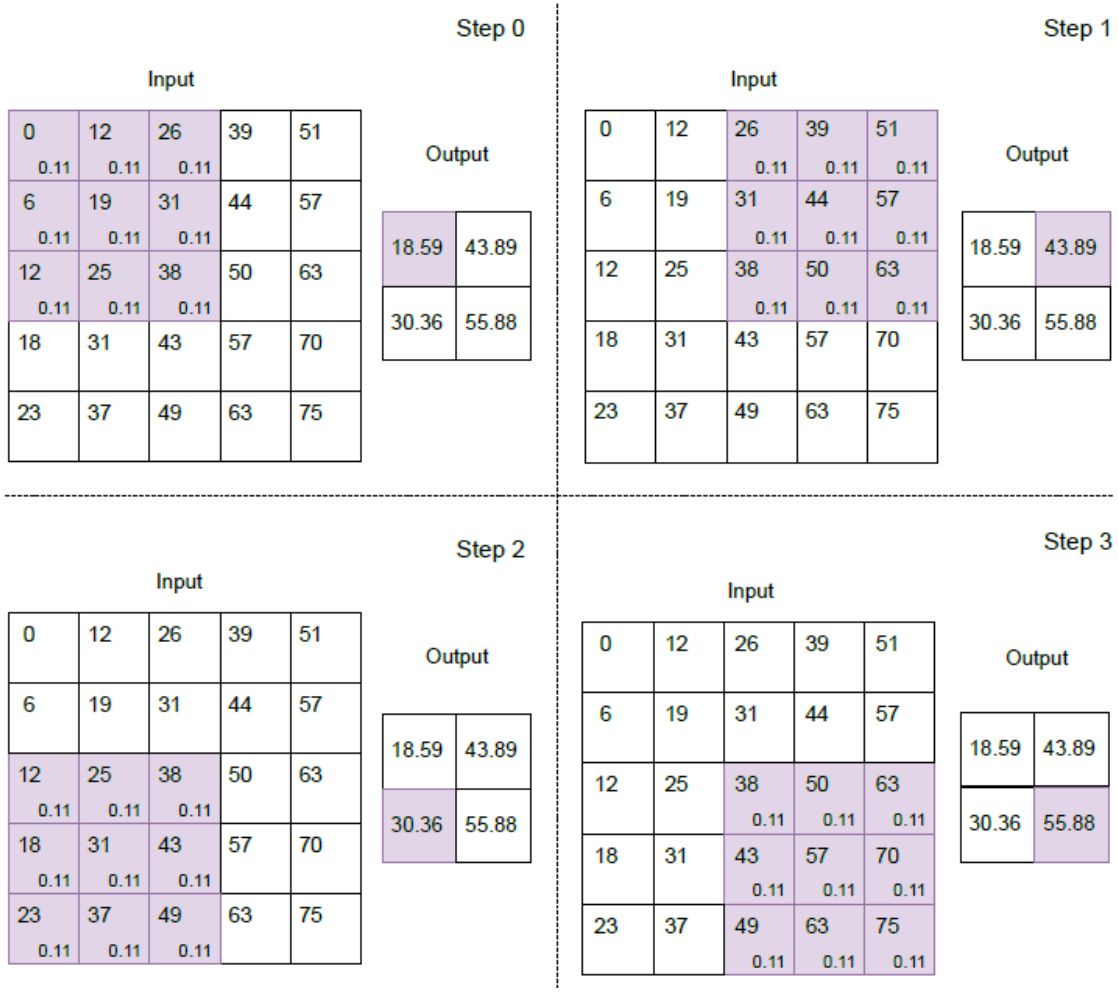
$$0*0+0*-1+0*0$$
$$+97*-1+96*5+99*-1$$
$$+101*0+102*-1+101*0=182$$

320	198	182			

Output Matrix

Source: Machinelearninguru.com/computer_vision/basics/convolution/convolution_layer.html

2D convolution with stride 2, valid padding



Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

SJSU SAN JOSÉ STATE UNIVERSITY

Convolutions on RGB images

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

W

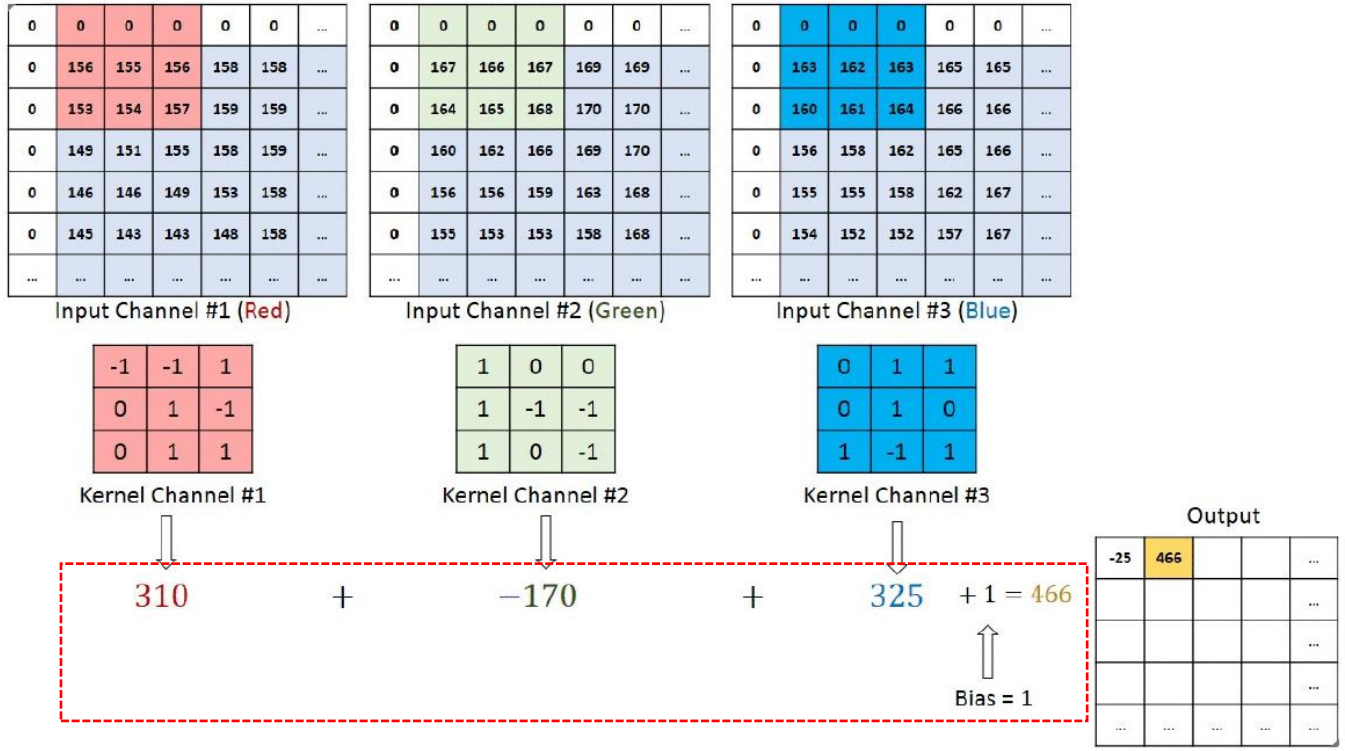
1	0	0
0	-1	0
0	0	1

Size: 3 x 3

Z = W*a(Red)+W*a(Green)+W*a(Blue)+b
(0-155+157)+(0-166+168)+(0-162+164)+5
= 1

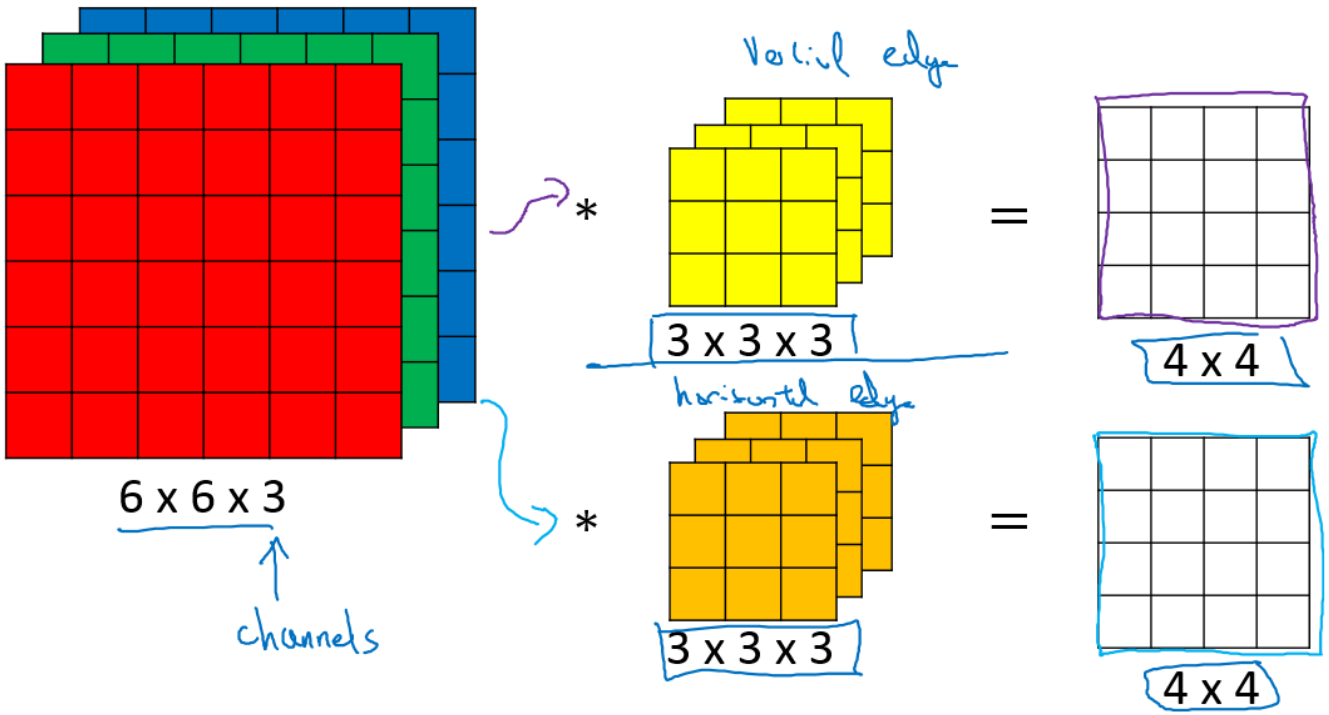
Source: Machinelearninguru.com/computer_vision/basics/convolution/convolution_layer.html

Convolutions on RGB images



Source: Machinelearninguru.com/computer_vision/basics/convolution/convolution_layer.html

Multiple filters (channels)

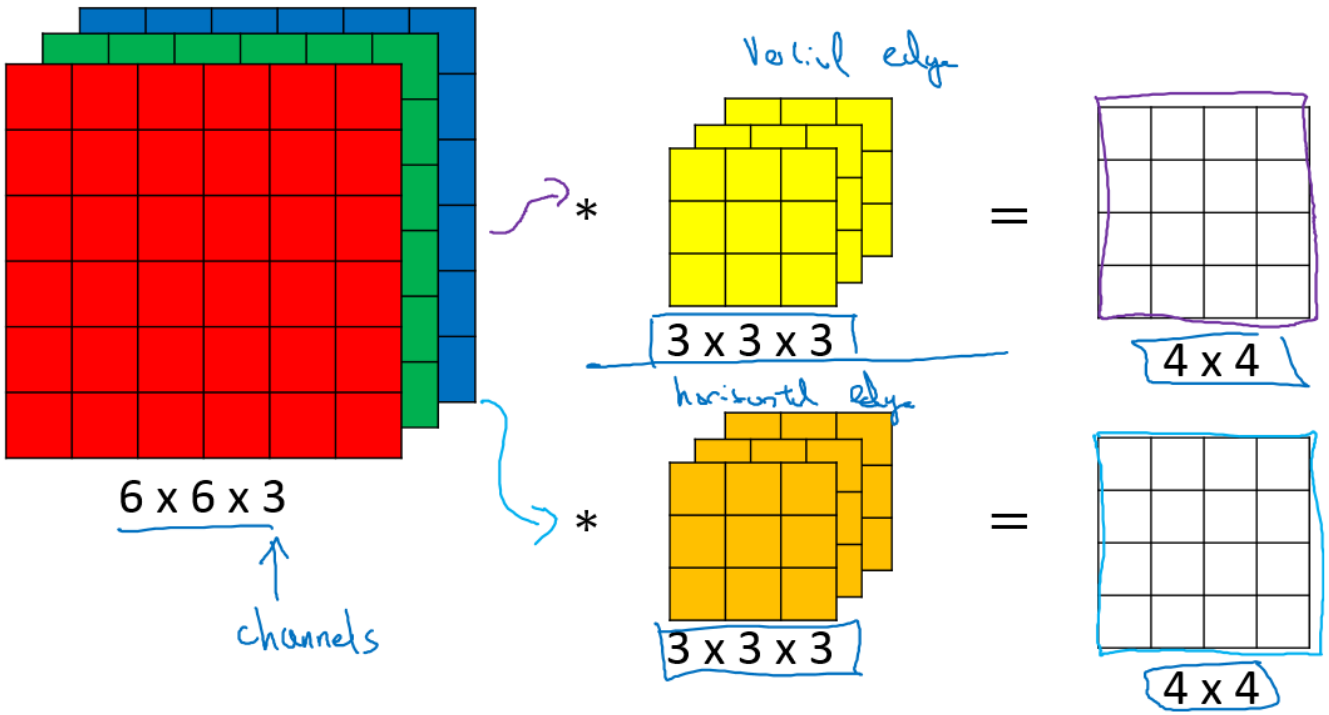


What is the shape of the kernel?

What is the shape of the output matrix?

Source: deep learning, Andrew Ng

Multiple filters (channels)



What is the shape of the kernel?

3x3x3x2

What is the shape of the output matrix?

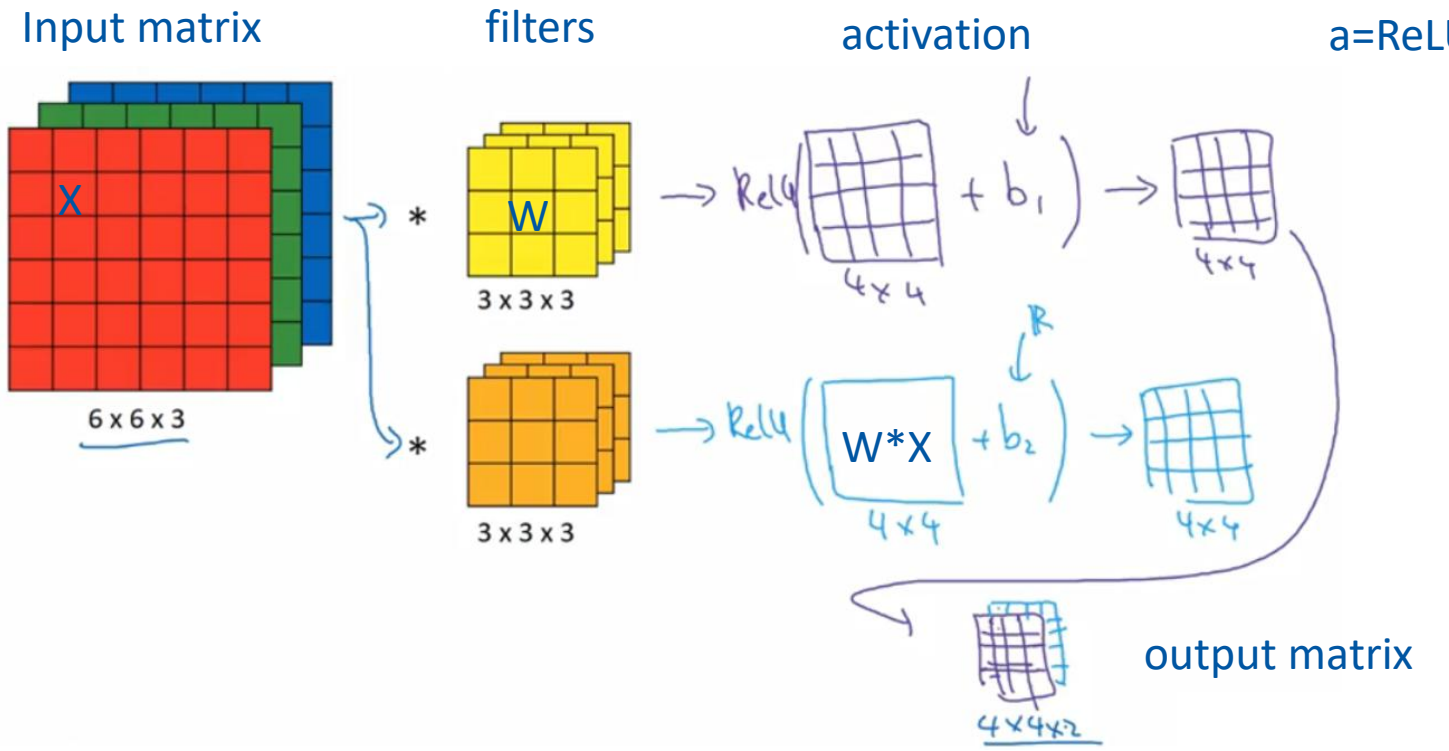
4x4x2

Source: deep learning, Andrew Ng

Multiple filters (channels)

Z=W*X + b

a=ReLU(Z)



Source: deep learning, Andrew Ng

Number of parameters in one layer

If there are 10 filters that have the size of 3 x 3 x 3 in one layer of a neural network, how many parameters does that layer have?

Number of parameters in one layer

If there are 10 filters that have the size of 3 x 3 x 3 in one layer of a neural network, how many parameters does that layer have?

- For 1 filter:
  3 x 3 x 3 = 27 for W
  1 for bias
- For 10 filters:
  Then, 28 * 10 = 280

Output size after convolution

- n x n image
- k x k filter
- padding p
- stride s

Output size:

$$\left[\frac{n+2p-k}{s}+1\right] \times \left[\frac{n+2p-k}{s}+1\right]$$

f is usually odd number in computer vision.

Number of parameters in a convolution layer

If *layer l* is a convolution layer:

$f^{[l]}$ = filter size

$p^{[l]}$ = padding

$s^{[l]}$ = stride

Input size: $n_H^{[l-1]} \times n_W^{[l-1]} \times \boxed{n_c^{[l-1]}}$

output size: $n_H^{[l]} \times n_W^{[l]} \times \boxed{n_c^{[l]}}$

$$n_H^{[l]} = \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1$$

$$n_W^{[l]} = \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1$$

Weights: $f^{[l]} \times f^{[l]} \times \boxed{n_c^{[l-1]}} \times \boxed{n_c^{[l]}}$

bias: $1 \times 1 \times 1 \times n_c^{[l]}$

Pooling

No parameters to learn

- To subsample (shrink) the input image in order to reduce the computational load, the memory usage, the number of parameters (reduce overfitting)

Pooling

2	3	1	9
4	7	3	5
8	2	2	2
1	3	4	5

Hyperparameters

f=2
s=2

Max:

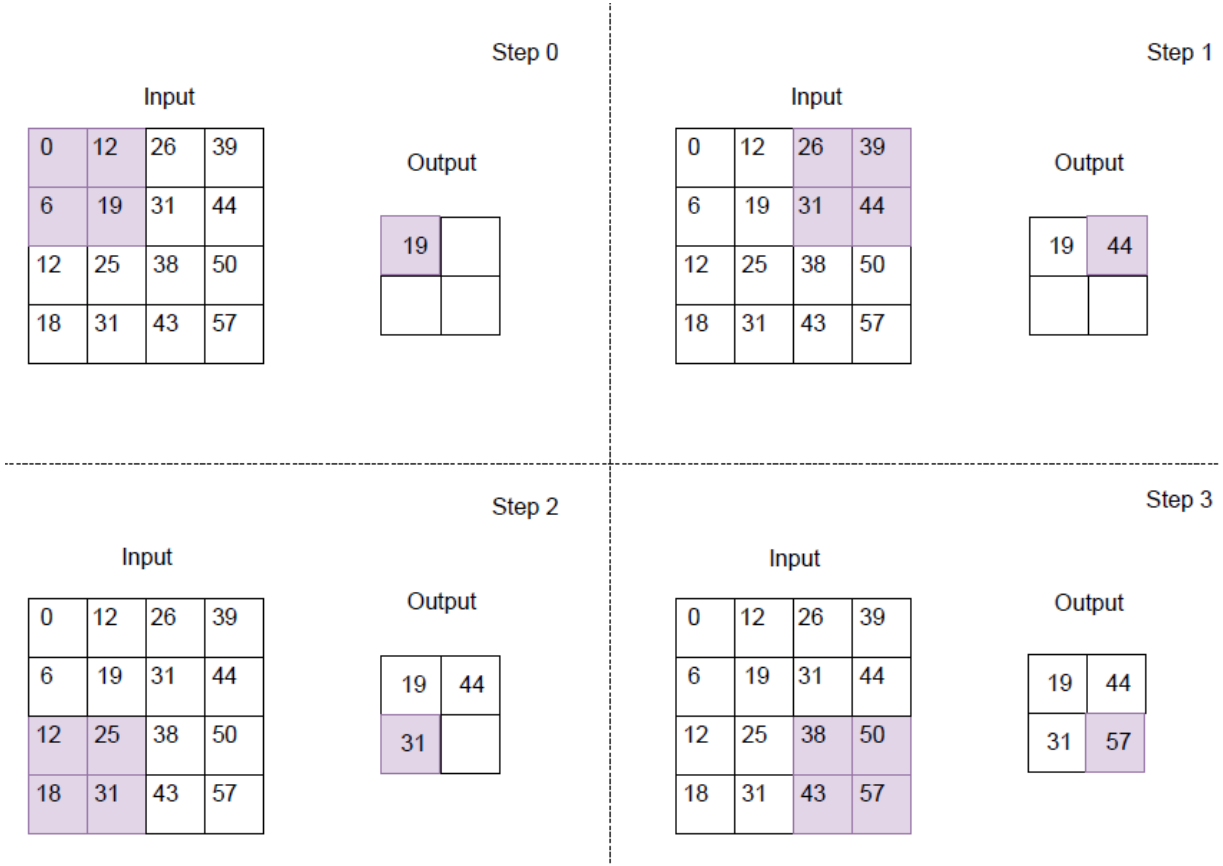
7	9
8	5

Average:

4	4.5
3.25	3.25

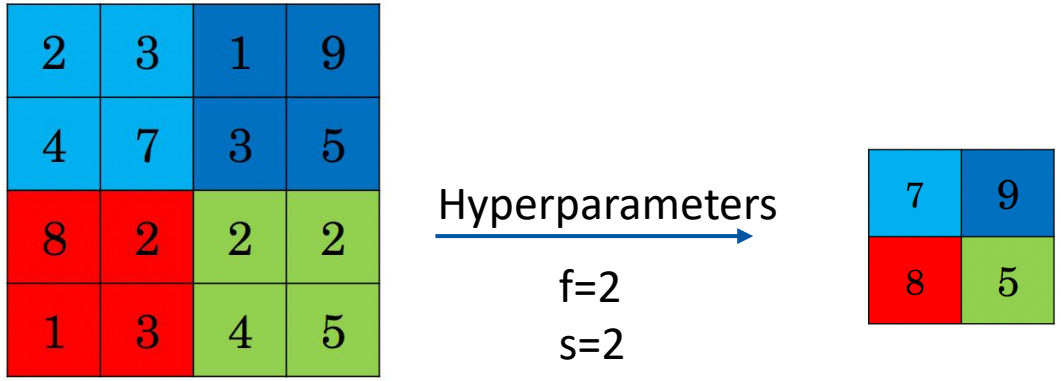
Source: Deep Learning, Andrew Ng

Max Pooling with 2x2 kernel and stride 2



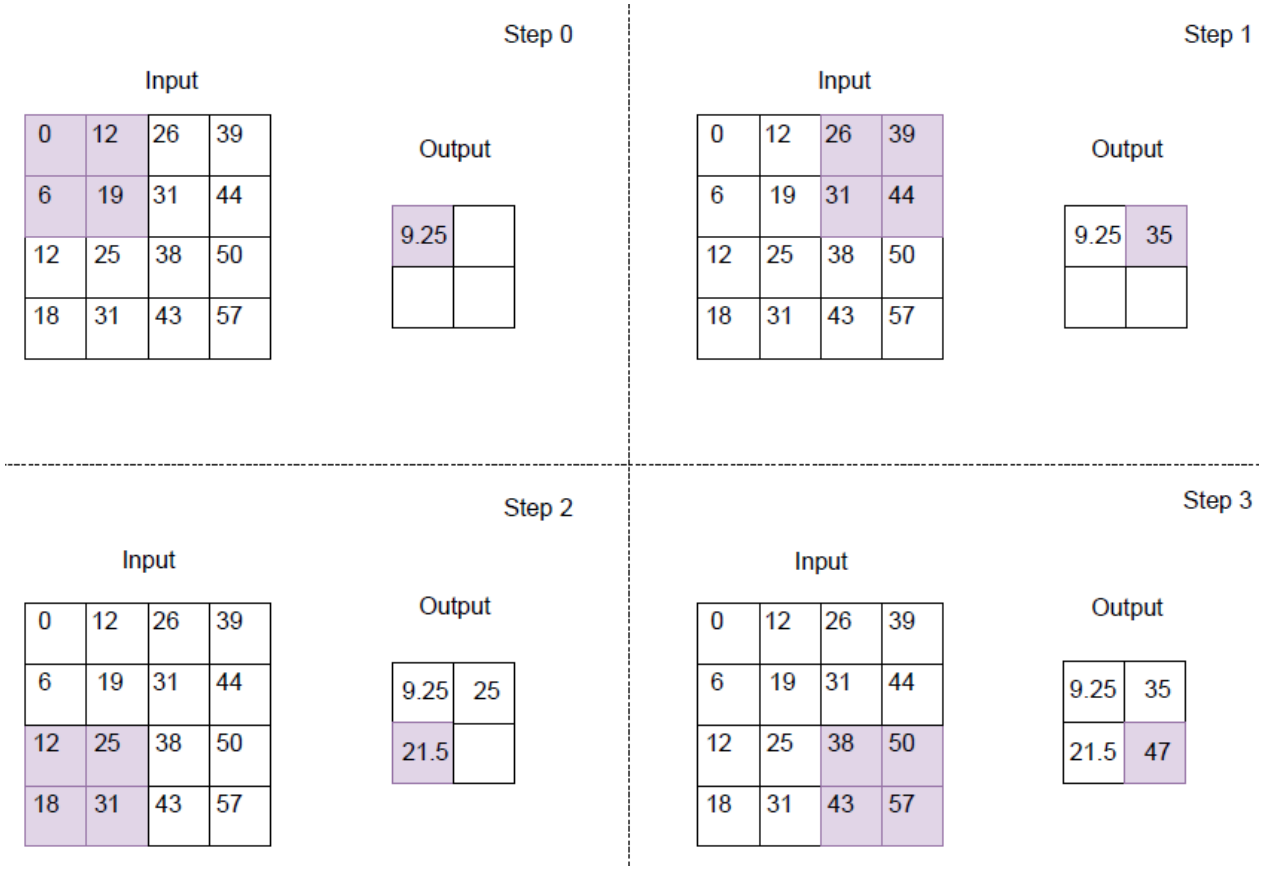
Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

Max Pooling



Source: Deep Learning, Andrew Ng

Average Pooling with 2x2 kernel and stride 2



Source: Math and Architectures of Deep Learning, Krishenedu Chauhury

SJSU SAN JOSÉ STATE UNIVERSITY

Average Pooling

2	3	1	9
4	7	3	5
8	2	2	2
1	3	4	5

Hyperparameters →

f=2

s=2

4	4.5
3.25	3.25

Source: Deep Learning, Andrew Ng

Size of Pooling

- Hyperparameters
  - f: filter size
  - s : stride
  - p : padding (usually p=0)

- Max or average pooling
- No parameters to learn

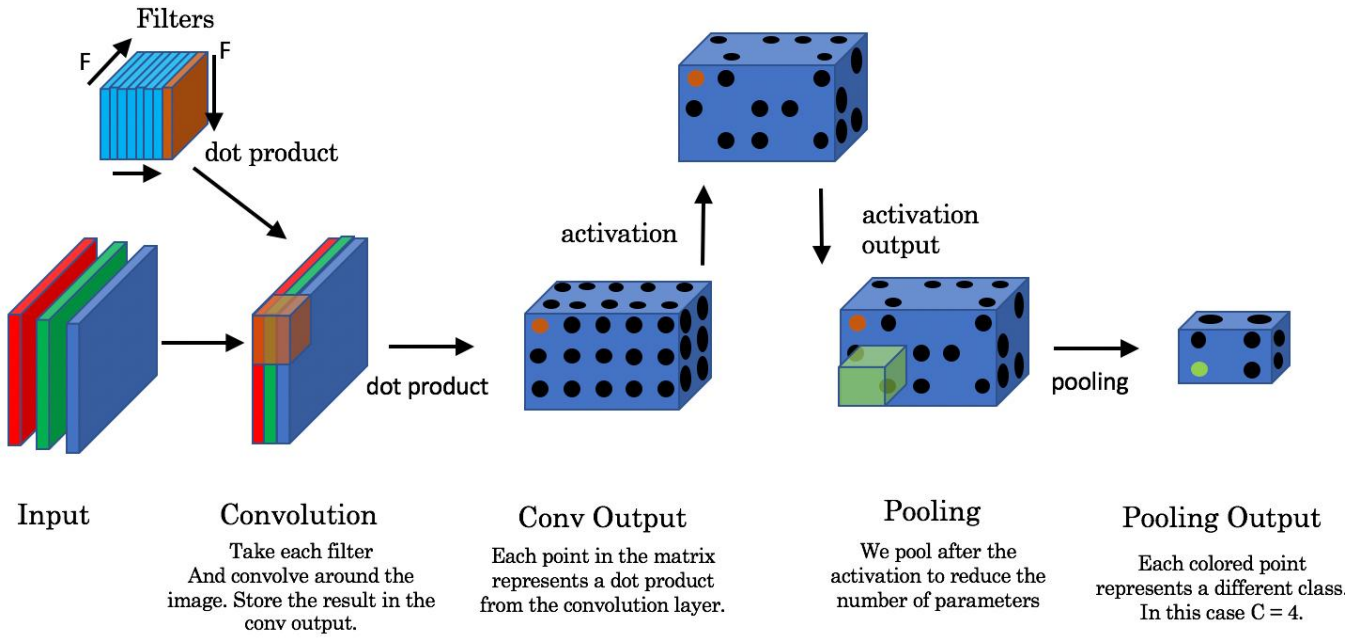
Input size: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$

output size: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

$$n_H^{[l]} = \frac{n_H^{[l-1]} - f^{[l]}}{s^{[l]}} + 1$$

$$n_W^{[l]} = \frac{n_W^{[l-1]} - f^{[l]}}{s^{[l]}} + 1$$

Convolution and Pooling



Source: Deep Learning, Andrew Ng